# Guía de estudio

Matemáticas para Inteligencia Artificial — UBP

Módulo 1 — Álgebra Lineal para la Representación de Datos

Módulo 2 — Cálculo Diferencial e Integral para IA

**Énfasis especial:** Derivadas, Integrales y la función de pérdida — cómo aprende realmente un modelo de IA.

Material de estudio - 2026

# Índice


1. Módulo 1 — Álgebra Lineal (resumen operativo)
1.1 Vectores, matrices y transformaciones lineales
1.2 Rango, núcleo, imagen y redundancia
1.3 Matrices especiales, autovalores y covarianza
1.4 Determinantes
2. Módulo 2 — Cálculo para IA
2.1 Funciones, continuidad y límites
2.2 DERIVADAS - en profundidad
2.3 INTEGRALES - en profundidad
3. FUNCIÓN DE PÉRDIDA - cómo aprende el modelo
3.1 Pérdida puntual, media y gradiente
3.2 Descenso por gradiente paso a paso
3.3 Ejemplo cerrado end-to-end
4. Glosario rápido

# 1. Módulo 1 — Álgebra Lineal (resumen operativo)

El Módulo 1 construye el lenguaje con el que la IA representa y transforma datos. En esta guía lo resumimos de forma operativa, porque el peso del estudio estará puesto en el Módulo 2 y en el proceso de aprendizaje por gradiente.

## 1.1 Vectores, matrices y transformaciones lineales

Un **vector** $x \in R^n$ es una lista ordenada de números. En IA representa datos (píxeles, embeddings, parámetros, estados). Una **matriz** $A \in R^{mxn}$ actúa como **operador lineal** $T_A(x) = A \cdot x$, es decir una capa densa: $z = W \cdot x + b$.

Multiplicación matriz-vector

$A \cdot x$ = combinación lineal de las columnas de A con coeficientes dados por x. La multiplicación matriz-matriz permite componer transformaciones y procesar un batch de ejemplos en paralelo ($Z = W \cdot X$).

## 1.2 Rango, núcleo, imagen y redundancia

El **rango** es el número máximo de columnas (o filas) linealmente independientes. Mide cuánta información conserva una transformación.

- $\text{rango}(A) \leq \min(m,n)$.
- A cuadrada invertible $\Leftrightarrow$ rango máximo $\Leftrightarrow \det(A) \neq 0$.
- Si rango < dim, la matriz **comprime** el espacio: hay pérdida de información.

El **núcleo** $\ker(A) = \{x : A \cdot x = 0\}$ contiene las direcciones que la transformación aplasta a cero. Si $\ker(A) = \{0\}$, la transformación es inyectiva. La **imagen** Im(A) es el conjunto alcanzado, y $\dim(\text{Im}(A)) = \text{rango}(A)$.

En redes neuronales, filas dependientes de W implican neuronas redundantes: la red aprende menos de lo que podría. Esto conecta la idea de **análisis vectorial** con la de **redundancia**.

## 1.3 Matrices especiales, autovalores y covarianza

- **Simétricas** ($A = A^T$): autovalores reales, autovectores ortogonales.
- **Definidas positivas** ($x^T \cdot A \cdot x > 0$): autovalores > 0, funciones convexas, mínimo único.
- **Ortogonales** ($Q^T \cdot Q = I$): preservan norma y ángulos; estabilidad numérica.
- **Dispersas** (sparse): mayoría de ceros; eficiencia en NLP y grafos.

Teorema espectral: $A = Q \cdot \Lambda \cdot Q^T$

Para matrices simétricas, Q es la base de autovectores ortogonales y $\Lambda$ la diagonal de autovalores. Un **autovalor** dice cuánto estira la matriz en una dirección; un **autovector** es esa dirección.

La **matriz de covarianza** es simétrica. Sus autovectores son las **direcciones principales** de los datos y sus autovalores miden la varianza en cada dirección. Esto es exactamente lo que hace PCA.

## 1.4 Determinantes

Escalar asociado a una matriz cuadrada que indica invertibilidad, cambio de volumen y estabilidad numérica. Métodos:

- 2x2: ad - bc.
- Sarrus (3x3): rápido y visual.
- Cofactores (nxn): general pero costoso.
- Reducción por filas (Gauss): triangular y producto de la diagonal; es la base numérica de NumPy/MATLAB.

# 2. Módulo 2 — Cálculo para IA

## 2.1 Funciones, continuidad y límites

Una función f: X → Y asigna a cada entrada x un único valor f(x). En IA se usan para modelar relaciones, activaciones y pérdidas. Es imprescindible dominar **dominio** (donde está definida), **codominio** (donde viven las salidas) e **imagen** (lo que efectivamente se alcanza).

### *Funciones típicas en IA:*

- Lineales y afines: $f(x) = w^T \cdot x + b$ (regresión y capas densas).
- Polinómicas: regresión polinómica y series de Taylor.
- Exponencial y logaritmo: softmax, entropía cruzada, log-verosimilitud.
- Sigmoide $\sigma(x) = 1/(1 + e^{-x})$ y tanh: activaciones acotadas.
- ReLU(x) = max(0, x) y variantes: estándar en redes profundas.

**Continuidad en a:** $\lim_{x \to a} f(x) = f(a)$. Intuitivamente, la gráfica se traza sin levantar el lápiz. Es necesaria para que pequeñas variaciones de los parámetros produzcan pequeños cambios en la pérdida.

### *Límites — herramientas clave*

El límite formaliza "a qué valor tiende f(x) cuando x se acerca a a". Definición ε-δ: $\lim_{x \to a} f(x) = L$ si para todo $\varepsilon > 0$ existe $\delta > 0$ tal que $0 < |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon$.

**Variantes:** límites laterales (⇒ el bilateral existe ssi ambos coinciden), límites infinitos (asíntotas verticales) y límites al infinito (asíntotas horizontales, relevante en sigmoide y tanh).

**Técnicas usuales:** sustitución directa, factorización ante 0/0, racionalización, división por la mayor potencia (límites al infinito).

# 2.2 DERIVADAS — en profundidad

## ¿Qué mide una derivada?

La derivada mide la **tasa de cambio instantánea** de la función en un punto. Geométricamente es la **pendiente de la recta tangente** a la gráfica en ese punto. En IA, esa pendiente es la señal que el algoritmo de entrenamiento usa para decidir **hacia dónde** mover un parámetro y **cuán rápido** hacerlo.

Definición formal

$$f'(a) = \lim_{h \to 0} [\, f(a + h) - f(a) \,] / h$$

El cociente (f(a+h) - f(a))/h es la pendiente de la **recta secante** entre dos puntos cercanos; al llevar h a 0, la secante se convierte en la tangente.

## Interpretación en IA (fundamental)

Si L(w) es la pérdida en función del parámetro w, entonces L'(w) es la pendiente del paisaje de error en ese punto. El algoritmo lee ese signo:

- L'(w) > 0 ⇒ la pérdida crece si aumentamos w ⇒ hay que **disminuir** w.
- L'(w) < 0 ⇒ la pérdida crece si disminuimos w ⇒ hay que **aumentar** w.
- L'(w) = 0 ⇒ punto crítico (candidato a mínimo).

## Derivadas laterales y derivabilidad

f es derivable en a si existen y coinciden las derivadas laterales $f'_+(a)$ y $f'_-(a)$. Ejemplo clásico: f(x) = |x| es continua en x = 0 pero no derivable allí (hay un pico). La relación general es **derivabilidad** ⇒ **continuidad**, pero la recíproca **no** vale.

En IA esto importa en ReLU, que no es derivable en 0; se trabaja con un **subgradiente** (típicamente 0) para no detener el entrenamiento.

## Reglas de derivación (indispensables para backprop)

| Regla | Expresión |
|---|---|
| Linealidad | $(\alpha f + \beta g)'(a) = \alpha \cdot f'(a) + \beta \cdot g'(a)$ |
| Producto | $(f \cdot g)'(a) = f'(a) \cdot g(a) + f(a) \cdot g'(a)$ |
| Cociente | $(f/g)'(a) = [f'(a) \cdot g(a) - f(a) \cdot g'(a)] / g(a)^2$ |
| Cadena | $(g \circ f)'(a) = g'(f(a)) \cdot f'(a)$ |

La **regla de la cadena** es, matemáticamente, la columna vertebral de la **backpropagation**: una red neuronal es una composición de funciones y el gradiente respecto de cada parámetro se obtiene multiplicando derivadas locales capa a capa.

## Derivadas elementales imprescindibles

| f(x) | f '(x) |
|---|---|

| | |
|---|---|
| c (constante) | 0 |
| x | 1 |
| $x^n$ | $n \cdot x^{n-1}$ |
| 1/x | $-1/x^2$ |
| √x | $1 / (2 \cdot \sqrt{x})$ |
| $e^x$ | $e^x$ |
| ln(x) | 1/x |
| sen(x) | cos(x) |
| cos(x) | -sen(x) |
| $\sigma(x) = 1/(1+e^{-x})$ | $\sigma(x) \cdot (1 - \sigma(x))$ |
| tanh(x) | $1 - \tanh^2(x)$ |
| ReLU(x) | 0 si x < 0; 1 si x > 0 (0 en x = 0) |

## Ejemplos resueltos

**Ejemplo 1 — Derivada por definición.** $f(x) = x^2$.

$$[f(a+h) - f(a)]/h = [(a+h)^2 - a^2]/h = (2ah + h^2)/h = 2a + h \to \mathbf{f'(a) = 2a}.$$

**Ejemplo 2 — Polinomio.** $f(x) = 3x^3 - 5x + 2$.

$$f'(x) = 9x^2 - 5.$$

**Ejemplo 3 — Regla de la cadena.** $f(x) = e^{2x+1}$.

$$f'(x) = e^{2x+1} \cdot 2 = 2 \cdot e^{2x+1}.$$

**Ejemplo 4 — Logaritmo compuesto.** $f(x) = \ln(x^2 + 1)$.

$$f'(x) = 2x / (x^2 + 1).$$

**Ejemplo 5 — Función a trozos.** $f(x) = x^2$ si $x < 0$; $f(x) = x$ si $x \geq 0$.

Derivada por la izquierda en 0: $\lim_{h \to 0■} (h^2 - 0)/h = 0$.
Derivada por la derecha en 0: $\lim_{h \to 0■} (h - 0)/h = 1$.
Como no coinciden, **f no es derivable en 0**, aunque sí es continua.

## Segunda derivada: curvatura y convexidad

f ''(x) describe cómo cambia la pendiente. Clasifica puntos críticos y determina la convexidad:

- f ''(x) > 0 ⇒ **convexa** (cóncava hacia arriba) — forma de U.
- f ''(x) < 0 ⇒ **cóncava** (hacia abajo) — forma de ∩.
- f '(x■) = 0 y f ''(x■) > 0 ⇒ **mínimo local**.
- f '(x■) = 0 y f ''(x■) < 0 ⇒ **máximo local**.

En IA, una pérdida convexa garantiza que todo mínimo local es también global. Por eso la pérdida cuadrática es la base del aprendizaje supervisado clásico: es convexa y se analiza en profundidad en la sección 3.

# 2.3 INTEGRALES — en profundidad

## Dos nociones unidas por el TFC

La integral tiene dos caras: la **indefinida** (familia de primitivas, operación inversa de la derivación) y la **definida** (área orientada bajo la curva en un intervalo [a,b]). El **Teorema Fundamental del Cálculo** conecta ambas.

## Integral indefinida — Primitiva

$$\int f(x)\,dx = F(x) + C, \text{ con } F'(x) = f(x).$$

La constante C aparece porque la derivada de una constante es cero: dos primitivas se diferencian a lo sumo en una constante.

| Función | Primitiva |
|---|---|
| x■ (n ≠ -1) | x■■¹ / (n+1) + C |
| 1/x | ln\|x\| + C |
| e■ | e■ + C |
| cos(x) | sen(x) + C |
| sen(x) | -cos(x) + C |
| Linealidad | ∫ [α·f + β·g] dx = α·∫f dx + β·∫g dx |

## Integral definida (Riemann)

Para f: [a,b] → R acotada, se construyen particiones P = {x■, x■, …, x■} y la suma de Riemann S(f, P, t■) = Σ f(t■)·Δx■. Si todas las sumas convergen al mismo número I cuando ||P|| → 0, se dice que f es Riemann-integrable y:

$$\int_a^b f(x)\,dx = I$$

Intuición: **área orientada bajo la curva**; cuenta positivo lo que está sobre el eje x y negativo lo que queda debajo.

## Propiedades operativas

- Linealidad: $\int [\alpha f + \beta g]\,dx = \alpha \cdot \int f\,dx + \beta \cdot \int g\,dx$.
- Aditividad: $\int_a^b = \int_a^c + \int_c^b$.
- Cambio de orientación: $\int_b^a = -\int_a^b$.
- Monotonía: si $f \le g$, entonces $\int f \le \int g$.
- Cota: $\left|\int_a^b f\right| \le M \cdot (b - a)$ si $|f| \le M$.

## Criterio de integrabilidad

**Toda función continua en [a,b] es Riemann-integrable.** Incluso funciones acotadas con un número finito de discontinuidades lo son. Por eso en IA basta con que la pérdida sea continua y

acotada en el intervalo relevante.

## Teorema Fundamental del Cálculo

**TFC I.** Si f es continua, $F(x) = \int_a^x f(t)\,dt$ es derivable y $F'(x) = f(x)$. Es decir, derivar el área acumulada devuelve la función integrando.

**TFC II (Regla de Barrow).** $\int_a^b f(x)\,dx = F(b) - F(a)$, con F primitiva de f. Esta es la fórmula que permite calcular integrales **sin** recurrir a sumas de Riemann.

## Técnicas clásicas

**Sustitución (cambio de variable).** Si u = g(x), entonces du = g'(x) dx:

$$\int 2x\cdot(x^2 + 1)^3\,dx, \text{ con } u = x^2 + 1 \Rightarrow \int u^3\,du = u^4/4 + C = (x^2 + 1)^4 / 4 + C.$$

**Integración por partes.** $\int u\,dv = u\cdot v - \int v\,du$.

$$\int x\cdot e^x\,dx, \text{ con } u = x,\ dv = e^x\,dx \Rightarrow v = e^x,\ du = dx$$
$$= x\cdot e^x - \int e^x\,dx = (x - 1)\cdot e^x + C.$$

## ¿Para qué sirven las integrales en IA?

Las integrales son el puente entre **pérdida puntual** y **pérdida media**. Cuando los datos se piensan como una distribución continua, la pérdida esperada sobre un intervalo (o toda la distribución) es una integral. También aparecen en probabilidades (densidades), entropías y en la verosimilitud marginal.

# 3. FUNCIÓN DE PÉRDIDA — cómo aprende el modelo

## La idea general

Un modelo de IA es, en el fondo, una familia de funciones parametrizadas $f_\theta$. Entrenarlo significa **buscar los parámetros $\theta$ que minimizan una función de pérdida L($\theta$)**, que mide cuánto se equivoca el modelo. La herramienta principal para esa búsqueda es el **descenso por gradiente**, que combina todo lo visto: continuidad (para que no haya saltos), derivadas (para saber hacia dónde moverse) e integrales (para promediar el error sobre los datos).

## 3.1 Pérdida puntual, pérdida media y gradiente

**Pérdida cuadrática puntual.** Para una predicción z y un valor real y:

$$\blacksquare(z) = (z - y)^2$$

Propiedades que la hacen ideal para aprender:

- **Dominio** = R, **Imagen** = [0, +∞): el error nunca es negativo.
- $\blacksquare(z) = 0 \Leftrightarrow z = y$ (acierto perfecto).
- $\blacksquare'(z) = 2 \cdot (z - y)$: el signo indica si el modelo sobrestima o subestima.
- $\blacksquare''(z) = 2 > 0$: **estrictamente convexa** ⇒ mínimo global único.
- De clase $C^\infty$: suave en todas partes, apta para gradientes.

### *Pérdida media (riesgo empírico).*

Sobre un conjunto de datos {($x\blacksquare$, $y\blacksquare$)}, i = 1..N, la pérdida cuadrática media (MSE) es:

$$MSE(\theta) = (1/N) \cdot \Sigma\blacksquare \left( f_\theta(x\blacksquare) - y\blacksquare \right)^2$$

En su versión continua (riesgo esperado) usamos una **integral**:

$$L(\theta) = \int \left( f_\theta(x) - y \right)^2 \cdot p(x, y)\, dx\, dy$$

Esta forma integral explica **por qué** necesitamos integrales en IA: para promediar la pérdida sobre todas las entradas posibles, no sólo sobre las observadas.

## Gradiente — generalización de la derivada

Cuando el modelo tiene varios parámetros $\theta = (\theta\blacksquare, \theta\blacksquare, \ldots, \theta\blacksquare)$, reunimos todas las derivadas parciales en un vector:

$$\nabla L(\theta) = \left( \partial L/\partial\theta\blacksquare\ ,\ \partial L/\partial\theta\blacksquare\ ,\ \ldots\ ,\ \partial L/\partial\theta\blacksquare \right)$$

El gradiente apunta en la dirección de **máximo crecimiento** de L. Para minimizar, nos movemos en la dirección **opuesta**.

## 3.2 Descenso por gradiente paso a paso

La regla de actualización es simple y es el motor del entrenamiento:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

donde $\eta$ **(eta)** es la **tasa de aprendizaje** (learning rate), un paso pequeño que decide cuánto nos movemos en cada iteración.

### *Interpretación geométrica (imaginá un valle)*

• Si L'(w) < 0, la pendiente baja hacia la derecha: debemos **aumentar** w.
• Si L'(w) > 0, la pendiente baja hacia la izquierda: debemos **disminuir** w.
• Si L'(w) = 0, no hay dirección preferida: estamos en un punto crítico.
El signo menos en $\theta - \eta \cdot \nabla L$ se encarga de invertir el gradiente para bajar.

### *El ciclo de entrenamiento*

| Paso | Qué ocurre matemáticamente | Herramienta clave |
|---|---|---|
| 1. Forward pass | Se calcula $f_\theta(x)$ para cada entrada. | Multiplicación matriz-vector |
| 2. Cálculo de pérdida | Se evalúa $\blacksquare(f_\theta(x), y)$ y se promedia. | Función + integral/suma |
| 3. Backward pass | Se computa $\nabla L(\theta)$ por regla de la cadena. | Derivadas y composición |
| 4. Actualización | $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$. | Descenso por gradiente |
| 5. Repetir | Hasta que la pérdida se estabilice. | Convergencia |

### *Elegir $\eta$ y leer la curva de pérdida*

• $\eta$ **muy pequeño**: entrenamiento lento, puede estancarse.
• $\eta$ **muy grande**: la pérdida oscila o diverge (se dispara).
• $\eta$ **adecuado**: la pérdida baja suavemente y se estabiliza en un mínimo.
La **curva de pérdida** (loss vs épocas) es el principal diagnóstico: si deja de bajar o vuelve a subir, suele indicar sobreajuste o $\eta$ mal calibrado.

# 3.3 Ejemplo cerrado end-to-end

Vamos a recorrer, con **todas las cuentas**, el proceso completo con el que un modelo lineal simple aprende. Este ejemplo integra límites, continuidad, derivadas, integrales y descenso por gradiente.

## Planteo del modelo

Modelo de regresión lineal unidimensional: ■(x) = θ·x.
Supongamos un valor objetivo fijo y■ = 2 que queremos reproducir para toda entrada x ∈ [0, 1] (uniformemente distribuida). La pérdida cuadrática media es:

$$L(\theta) = \int_0^1 (\theta \cdot x - 2)^2 \, dx$$

## Paso 1 — Expresar L(θ) usando integrales

Desarrollamos el cuadrado:

$$(\theta \cdot x - 2)^2 = \theta^2 \cdot x^2 - 4 \cdot \theta \cdot x + 4$$

Aplicamos linealidad de la integral y calculamos cada pieza:

$$\int_0^1 x^2 \, dx = 1/3$$
$$\int_0^1 x \, dx = 1/2$$
$$\int_0^1 1 \, dx = 1$$

Sustituyendo:

$$L(\theta) = \theta^2 \cdot (1/3) - 4\theta \cdot (1/2) + 4 = \theta^2/3 - 2\theta + 4$$

Observación: la integral convirtió un problema "puntual" (perder (θx - 2)² para cada x) en una **función real explícita de** θ que ya podemos derivar y analizar.

## Paso 2 — Analizar L(θ) con derivadas

Derivamos respecto de θ:

$$L'(\theta) = 2\theta/3 - 2$$

Punto crítico (L'(θ) = 0):

$$2\theta/3 = 2 \Rightarrow \theta^* = 3$$

Segunda derivada:

$$L''(\theta) = 2/3 > 0 \Rightarrow L \text{ es estrictamente convexa.}$$

Por lo tanto θ* = 3 es el **mínimo global único**, y al evaluar: L(3) = 9/3 - 6 + 4 = 1.

## Paso 3 — Verlo como descenso por gradiente

Supongamos que empezamos con θ■ = 0 y usamos tasa de aprendizaje η = 0.9. Aplicamos repetidamente $\theta_{t+1} = \theta_t - \eta \cdot L'(\theta_t)$.

| t | θ_t | L'(θ_t) = 2·θ/3 - 2 | θ_{t+1} = θ_t - 0.9·L'(θ_t) | L(θ_t) |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 0 | 0.000 | -2.000 | 0.000 - 0.9·(-2.000) = 1.800 | 4.000 |
| 1 | 1.800 | -0.800 | 1.800 - 0.9·(-0.800) = 2.520 | 1.480 |
| 2 | 2.520 | -0.320 | 2.520 - 0.9·(-0.320) = 2.808 | 1.077 |
| 3 | 2.808 | -0.128 | 2.808 - 0.9·(-0.128) = 2.923 | 1.012 |
| 4 | 2.923 | -0.051 | 2.923 - 0.9·(-0.051) = 2.969 | 1.002 |
| 5 | 2.969 | -0.020 | 2.969 - 0.9·(-0.020) = 2.988 | 1.000 |
| … | → 3.000 | → 0 | estaciona en θ* = 3 | → 1 (mínimo) |

Observá que el gradiente arranca grande y en cada paso se achica: el modelo frena solo cuando está cerca del óptimo. Esto es la forma matemática del **aprendizaje**.

## Paso 4 — Qué pasa si cambia η

• Con η = 0.1 el modelo también converge, pero necesita muchas más iteraciones.
• Con η = 3 (demasiado grande) el modelo **sobrepasa** el mínimo y oscila; si η es aún mayor, la pérdida **diverge**.
• La curvatura L''(θ) = 2/3 nos informa cuán agresiva puede ser η: en el límite, η < 2 / L''(θ) = 3 asegura convergencia.

## Síntesis del proceso de aprendizaje

1. **Definir $f_\theta$** (el modelo): función parametrizada.
2. **Definir ■(z, y)** (pérdida puntual): mide error de una predicción.
3. **Agregar sobre los datos** (suma o integral): construir L(θ), riesgo promedio.
4. **Derivar L(θ)** respecto de cada parámetro para obtener ∇L(θ).
5. **Actualizar** θ ← θ - η·∇L(θ), repetir.
6. **Verificar**: estudiando L'(θ), L''(θ) y comportamiento asintótico sabemos si existe un mínimo y si el método converge.

# 4. Glosario rápido

| | |
|---|---|
| **Autovalor / Autovector** | Cuánto estira la matriz / dirección en la que lo hace. |
| **Backpropagation** | Cálculo de gradientes capa a capa aplicando la regla de la cadena. |
| **Convexidad** | f''(x) ≥ 0; garantiza mínimo global único. |
| **Derivada** | Pendiente de la tangente; tasa de cambio instantánea. |
| **Derivada parcial** | Derivada tratando a las demás variables como constantes. |
| **Descenso por gradiente** | Regla iterativa θ ← θ - η·∇L. |
| **Función de pérdida** | Mide el error del modelo; es lo que se minimiza. |
| **Gradiente** | Vector de derivadas parciales; indica el máximo crecimiento. |
| **Integral definida** | Área orientada bajo la curva en [a,b]. |
| **Integral indefinida** | Familia de primitivas; inversa de la derivación. |
| **Learning rate (η)** | Tamaño del paso en el descenso por gradiente. |
| **Límite** | Valor al que se aproxima f(x) cuando x → a. |
| **MSE** | Error cuadrático medio: (1/N)·Σ (■■ - y■)². |
| **PCA** | Reducción de dimensionalidad usando autovectores de la covarianza. |
| **Rango** | Dimensión del subespacio generado por filas/columnas. |
| **Regla de la cadena** | (g ■ f)'(a) = g'(f(a))·f'(a); base de backprop. |
| **TFC (Barrow)** | ∫■■ f dx = F(b) - F(a). |

*Fin de la guía. El objetivo es que, al terminar de leerla, puedas mirar cualquier modelo de IA y reconocer debajo el mismo esquema: una función parametrizada, una pérdida, un gradiente y una actualización iterativa.*